

TailGuard: Testing Joint Placement and Transfer Control for Pooled KV Memory

Anonymous Authors

Abstract—Pooling host memory across prefill and decode nodes can retain more KV state, yet remote hits compete with refill and migration for shared transfer resources. TailGuard studies whether object placement and request-aware transfer budgets can reduce exposed remote-read tails under a fixed total memory budget. It is a policy plugin for PegaFlow, whose storage and transfer engine remains an independent dependency. The proposed controller couples placement, demand-read admission and background transfer pacing, while preserving object identity and lifecycle correctness. Strong controls include tuned static placement and transfer budgets, PegaFlow without the policy, and supported disaggregated-cache systems with advisory prefetch. The first milestone must establish a repeatable capacity-versus-tail conflict before policy implementation. This repository contains a research proposal only: no policy implementation, RDMA trace, NPU validation or performance result is reported.

I. PROBLEM AND NECESSITY

Can joint P/D host-memory placement and transfer admission reduce request p99 when remote reads compete with refill/migration, at fixed total bytes and useful completion rate? Long-context serving can exchange recomputation for a remote hit whose lookup, queue, DMA and materialization delay exceeds its apparent cache benefit. Such a conflict is a hypothesis until matched traces expose it.

II. PRIOR WORK AND SCOPE

SYMPHONY already uses advisory prefetch and cooperative cache management; Mooncake already pools disaggregated storage. The proposed contribution must exceed tuned memory/priority/transfer controls, not claim disaggregation or advisory prefetch as new. See SYMPHONY and Mooncake [1], [2]. PegaFlow remains the engine; this repository studies a policy plugin, with separate evidence custody.

III. MECHANISM HYPOTHESIS

Use object identity, estimated read deadline and measured queue/transfer pressure to jointly choose placement and background transfer budgets. Admit demand reads without allowing refill or migration to consume their entire service window; account for deferred and rejected work. Implement later as a PegaFlow policy plugin with explicit control hooks. Keep the PegaFlow engine, RDMA transport, packing/layout operators and runtime lifecycle contract separate. A source/trace hook feasibility audit is required before a controller.

IV. MATCHED EVALUATION

Compare decode-local cache, pooled static placement, best static priority/budget, PegaFlow without TailGuard, and compatible SYMPHONY/Mooncake controls. Match models, arrivals, completed work, total CPU/GPU memory, topology

and bandwidth. Report hit/useful-residence, remote-read p99, request p99, queue waits, bytes, fairness and recomputation. Compare placement only, pacing only and the joint policy. A future-information oracle is headroom, not a deployable baseline. Fix precision and charge delayed/refused work, metadata, CPU and all transfers.

V. STOPPING RULES

Stop if pooling adds no useful residence, shared transfers create no repeatable tail conflict, the best static policy matches the mechanism, or lower p99 comes from rejecting/deferring more work. No artificial TTL increase counts as benefit. All proposed effects remain unverified. No runtime implementation or experiment is reported.

REFERENCES

- [1] S. Agarwal, B. Hu, A. Mao, A. Akella, and S. Venkataraman, “SYMPHONY: Enabling Compute-Memory disaggregation in LLM serving systems,” in *23rd USENIX Symposium on Networked Systems Design and Implementation*, 2026, pp. 2027–2041. [Online]. Available: <https://www.usenix.org/conference/nsdi26/presentation/agarwal>
- [2] R. Qin, Z. Li, W. He, M. Zhang, Y. Wu, and W. Zheng, “Mooncake: Trading more storage for less computation—a kvcache-centric architecture for serving llm chatbot,” in *23rd USENIX Conference on File and Storage Technologies*, 2025, pp. 155–170. [Online]. Available: <https://www.usenix.org/conference/fast25/presentation/qin>